

# Enhancing Next POI Recommendations with Cluster-Aware Graph Attention Networks

Fabrizio Petroni, Davide Pintus

## Abstract

In this work, we propose a deep learning architecture that improves next Point-of-Interest (POI) category prediction based on historical user check-ins. Our model is built using five different modules: (1) an embedding module that encodes discrete spatio-temporal and categorical information, (2) a Local Pattern Learning module that models intra-cluster transitions using Graph Convolutional Networks, (3) a General Pattern Learning module that captures inter-cluster dynamics via Graph Attention Networks, (4) a Spatio-Temporal Aware LSTM to model sequential dependencies in time and space, and (5) a Candidate POI Matching module that applies cross-attention to align the aggregated patterns with candidate POIs. We train and evaluate our model using Tokyo's check-in from the Foursquare dataset.

## Keywords

Point Of Interest (POI), K-Means, Trajectory Flow Graph, Graph Convolution, Graph Attention, Spatio-Temporal LSTM, POI Recommendation, *Recall@K*

## 1. HIGHLIGHTS

- Introduces a novel architecture, Cluster-Aware Graph Attention Networks, for next POI category prediction.
- Integrates both local and global pattern learning via graph convolutional and graph attention networks, respectively.
- Incorporates a spatio-temporal aware LSTM to effectively capture sequential dependencies in user trajectories.
- Leverages a candidate POI matching module that uses cross-attention to align learned patterns with potential POI categories.
- Demonstrates improved recommendation performance on the Foursquare Tokyo dataset compared to state of the art baselines.

## 2. Introduction

Location-based services play an important role in our daily urban life. They affect how we move in our cities and find new places to eat, shop, or relax. Thanks to the rise in mobile devices and cloud technology, we now have access to a huge amount of check-in data from services like Foursquare. This data, which includes location information, time stamps, and other details, helps us understand how people move around over time and space [1].

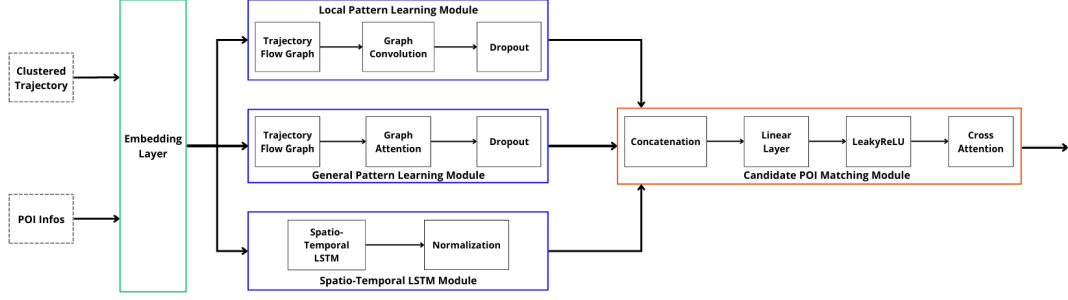
Early research on POI recommendation mostly focused on methods that looked for known movement patterns in users' past check-in data to predict their future visits. Methods like Markov chains and matrix factorization were common because they treated the problem as one of predicting the next item in a sequence, and they often produced good results. With the rise of deep learning, many

studies began using techniques such as convolutional neural networks (CNNs), Long Short-Term Memory networks (LSTMs) [2], gated recurrent units (GRUs), and self-attention mechanisms to better understand human mobility. These deep learning approaches have led to significant improvements in POI recommendation tasks, both in research and in practice. More recent work has also started to add spatio-temporal correlations into the models, which makes the predictions even more accurate.

In this project, we propose a new system specifically designed for predicting the next POI category that a user is likely to visit. Our model is structured into five main parts. First, our embedding module converts raw spatio-temporal and categorical data into continuous vector representations. Next, a local pattern learning module uses Graph Convolutional Networks (GCNs) [3] to capture behaviour within groups of nearby POIs, while a general pattern learning module employs Graph Attention Networks (GATs) [4] to model movements between different POI clusters. We further incorporate a spatio-temporal LSTM that learns the sequential order and timing of check-ins. Finally, a candidate POI matching module uses a Cross-Attention mechanism [5] to combine these insights, ultimately predicting the most likely POI category.

By decomposing the problem into these focused components, our approach is able to leverage both local and global movement patterns along with the sequential and temporal aspects of user behaviour. This results in a more accurate and robust recommendation system. We evaluated our model using the Foursquare Tokyo dataset, and our experiments indicate that it outperforms several state of the art methods, particularly when considering *Recall@1*, where our predictions achieve exceptional precision.

The rest of the project is organized as follows. First, we



**Figure 1:** The proposed Cluster-Aware Graph Attention Network architecture underlines the integration of local and general pattern learning with spatio-temporal modelling for better POI recommendation.

provide a detailed review of related work in POI recommendation and mobility prediction. Then, we describe our proposed system and its implementation. Following that, we detail our experimental setup and results, including quantitative metrics and comparisons with existing methods. Finally, we conclude with a discussion of our findings and potential avenues for future research.

### 3. Related Works

The problem of recommending the next POI was addressed with the use of various deep learning methods.

The use of Graph Neural Networks (GNNs) makes possible to derive spatial information by highlighting links between neighbouring POIs. Spatial information is important in the selection of subsequent POIs, as highlighted in [6]. Various methods can be used to analyse this information, such as Graph Convolution [7] and Graph Attention [4]; in the first case no weights are applied to the nodes, while in the second case the closest nodes have a greater weight than the most distant ones.

Another valuable tool for this kind of task are the Recurrent Neural Networks (RNNs), which have performed very well. This is because these kind of networks are able to store past information extrapolated from past inputs, allowing them to be used with current inputs to determine the next POI. This has been demonstrated in previous studies, such as in the case of the DeepMove [8], GESTIAN [9] and ATST-LSTM [10] models. Moreover, the use of a spatio-temporal version of such architectures, as seen in [9] and [11], allows to capture spatio-temporal correlations in the user’s current trajectory, allowing for further performance improvement.

### 4. Model

The proposed model is shown in figure 1. It is composed by five modules:

1. Embedding Module;

2. Local Pattern Learning Module;
3. General Pattern Learning Module;
4. Spatio-Temporal Aware LSTM;
5. Candidate POI Matching Module.

Module (1) computes the embedding of user trajectories received as input. Modules (2), (3) and (4) work in parallel and receive as input the embedding of specific information extracted from the input trajectories. Module (5) fuses the features extracted from the three previous modules and through cross attention determines the probability of each possible category for the next POI.

Model’s objective is to determine the category of the next POI in which a user, whose past trajectory is known, will move; to do this, the model extracts 3 different types of features: the movement patterns between POIs in the same cluster using the Local Pattern Learning Module, the movement patterns between POIs in different clusters using the General Pattern Learning Module and short- and long-term spatio-temporal dependencies of user movements using the Spatio-Temporal Aware LSTM.

A description of the variables that will be mentioned in this section is given in tables below:

| Common model components                  |                                                                                                                        |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| $W_*, U_*, O_*$                          | Weight matrices                                                                                                        |
| $\sigma, softmax, tanh, LeakyReLU$       | Activation functions                                                                                                   |
| $b_*$                                    | Bias terms                                                                                                             |
| Embedding Module                         |                                                                                                                        |
| $\Delta$                                 | Correlation matrix between the position of a POI and the check-in time at which it was visited by the considered users |
| $p_i, t_i, \Delta_{ij}^p, \Delta_{ij}^t$ | Position/time of the $i$ -th check-in and pairwise differences between the $i$ -th and $j$ -th check-ins.              |
| $p_{max}, p_{min}, t_{max}, t_{min}$     | Max/min position (p) and time (t) in considered batch                                                                  |

| Local (L) and General (G) Pattern |                                                                                                      |
|-----------------------------------|------------------------------------------------------------------------------------------------------|
| $k$                               | Minimum transitions needed to form an edge ( $k = 1$ )                                               |
| $A$                               | POI connection matrix (with self-loops: $A + I$ ).                                                   |
| $D$                               | Diagonal matrix of row sums from $A + I$ ; used for normalization.                                   |
| $H^{(l)}$                         | Node features at layer $l$ (initially POI embeddings).                                               |
| $h_{gen,i}$                       | Feature vector for node $i$ in the attention mechanism.                                              |
| $\alpha_{ij}$                     | Normalized attention weight of node $i$ for node $j$ .                                               |
| $L, G$                            | Aggregated node features: L = local average within each cluster; G = global weighted across clusters |

| Spatio-Temporal Aware LSTM (S_T) |                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------------------|
| $x_t$                            | Input vector                                                                                    |
| $m_{ij}$                         | Spatio-temporal embedding                                                                       |
| $f_t, i_t, o_t$                  | Spatio-temporal forget ( $f_t$ ), input ( $i_t$ ) and output ( $o_t$ ) gates at time step $t$ . |
| $\tilde{c}_t$                    | Candidate memory cell at time step $t$                                                          |
| $c_t$                            | Memory cell at time step $t$                                                                    |
| $h_t$                            | Hidden state at time step $t$                                                                   |

| Candidate POI Matching |                                                                      |
|------------------------|----------------------------------------------------------------------|
| $features, F$          | Feature vectors before and after applying LeakyReLU in the MLP layer |
| $D$                    | Embedding of candidate POI                                           |
| $Q, K, V$              | Cross Attention's Query, Key and Value                               |
| $d$                    | Last dimension of tensor $Q$                                         |

**Table 1**  
Description of variables used in the model section, divided by modules.

## 4.1. Embedding Module

This is the first module of the model; its task is to transform the discrete sequences received as input into continuous sequences. Specifically, the module receives two packs as input: the first is called data ( $num\_users, seq\_length, 6$ ) and contains the information concerning the check-ins performed by the  $N$  users considered; the second one is the poi\_infos, which contains the positions and categories of all the POIs in the dataset considered.

Given these inputs, the module returns five outputs. First, the combined\_embedding is produced combining the embeddings obtained from the six feature groups contained in 'data'. Second, the poi\_category\_ids\_embedding corresponds to the embed-

ding of the POIs categories visited by the users considered in the given sequences. Third, the poi\_position\_embed is the embedding of the categories of the POIs located closest to the POI where the given users are currently located. Fourth, the candidate\_poi\_embed represents the embedding of the categories of the POIs located closest to the POI where the given users are currently located. Finally, the diff\_embed encodes a correlation matrix between the position of a POI and the check-in time at which it was visited by the considered users; such matrix is defined as:

$$\Delta = \begin{bmatrix} \Delta_{11} & \Delta_{12} & \dots & \Delta_{1n} \\ \Delta_{21} & \Delta_{22} & \dots & \Delta_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \Delta_{m1} & \Delta_{m2} & \dots & \Delta_{mn} \end{bmatrix} \quad (1)$$

Here,  $m$  and  $n$  are the batch size (number of users) and the sequence length (number of check-ins), and  $\Delta_{ij} = [\Delta_{ij}^p, \Delta_{ij}^t]$ .  $\Delta_{ij}^p$  and  $\Delta_{ij}^t$  are computed as:

$$\Delta_{ij}^p = \frac{|p_i - p_j|}{p_{max} - p_{min}} \quad (2)$$

$$\Delta_{ij}^t = \frac{|t_i - t_j|}{t_{max} - t_{min}} \quad (3)$$

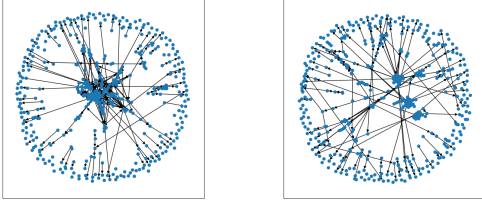
## 4.2. Local and General Pattern Learning

This modules aim to capture local and general patterns in user movements. They consist of two components: construction of the Trajectory Graph and Local Pattern Learning/General Pattern Learning.

### 4.2.1. Trajectory Flow Graph

To construct the trajectory flow graph  $G$ , all POIs that appear in the historical trajectories of users within a batch are included as nodes. Each node's initial feature is formed by combining its location embedding and category embedding, and considers also its label embedding. For each user's check-in sequence, a transition from  $v_i$  to  $v_j$  represented as a directed edge in the graph. Each module constructs its own trajectory graph. The definition of the nodes is identical for both modules. A directed edge from  $v_i$  to  $v_j$  is added if:

- Local Pattern: at least  $k$  user go to  $v_i$  to  $v_j$  and  $v_i$  and  $v_j$  belong to the same cluster  $c$ ; see figure 2.
- General Pattern: at least  $k$  user go to  $v_i$  to  $v_j$  and  $i$  and  $j$  belong to different clusters  $c_u$  and  $c_v$ ; see figure 2.



**Figure 2:** Local pattern (left) and general pattern (right) graph examples; in both cases we used  $k = 1$ , because the number of check-ins per user was too low to use a value greater than 1

#### 4.2.2. Local Pattern Learning

The local pattern learning module captures intra-cluster user behaviour patterns. Given the Trajectory graph  $G = (V, E)$ , Graph Convolutional ([7]) layers are applied to aggregate neighbourhood information and update node representations, following the approach of Kipf and Welling [3]. The feature propagation is defined as:

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W_{loc}^{(l)}) \quad (4)$$

where  $\tilde{A} = A + I$  is the adjacency matrix with self-loops,  $\tilde{D}$  is its degree matrix,  $H^{(l+1)}$  is the node feature matrix at layer  $l + 1$ ,  $W^{(l)}$  is the learnable weight matrix, and  $\sigma$  is the activation function. The final local pattern representation is obtained by averaging all node features:

$$\mathbf{L} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathbf{x}_v \quad (5)$$

#### 4.2.3. General Pattern Learning

The general pattern learning module captures cross-cluster mobility behavior patterns. To learn node-level representations, we apply a GAT [4], which allows the model to assign varying importance to neighboring nodes during the message-passing process. For a given node  $i$ , the output is computed as:

$$h'_{gen,i} = ||_{h=1}^H \sigma \left( \sum_{j \in \mathcal{N}} \alpha_{gen,i,j}^{(h)} W_{gen}^{(h)} h_{gen,j} \right) \quad (6)$$

where  $h_j$  is the feature vector of the node  $j$ ,  $W$  is a learnable linear transformation,  $\sigma$  is an activation function,  $\alpha_{ij}$  is the attention coefficient, computed as:

$$a_{ij} = \frac{\exp(\text{LeakyReLU}(W_{gen}^T \cdot [h_{gen,i} || h_{gen,j}]))}{\sum_{k=1}^n \exp(\text{LeakyReLU}(W_{gen}^T \cdot [h_{gen,i} || h_{gen,j}]))} \quad (7)$$

The final general pattern representation is obtained by averaging all node features:

$$\mathbf{G} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathbf{x}_v \quad (8)$$

### 4.3. Spatio-Temporal Aware LSTM

The LSTM is a RNN that allows us to capture the long-term dependencies of the input sequences [2]. In this case, we are interested in capturing long-term spatio-temporal dependencies [9, 11]; in order to achieve this, it was necessary to adapt the original architecture of the LSTM by modifying its gates to incorporate the spatio-temporal embedding ( $m_{ij}$ ) as an additional input, as done for the GESTIAN model with its spatio-temporal GRU in [9]. The forget gate, which determines what information to discard from the previous cell state  $c_{t-1}$ , is defined as follow:

$$f_t = \sigma(W_f \cdot x_t + U_f \cdot h_{t-1} + O_f \cdot m_{ij} + b_f) \quad (9)$$

Similarly, the input gate controls how much of the current input  $x_t$  must be stored in the cell state  $c_t$ , and is computed as:

$$i_t = \sigma(W_i \cdot x_t + U_i \cdot h_{t-1} + O_i \cdot m_{ij} + b_i) \quad (10)$$

Finally, the output gate controls which part of the cell state  $c_t$  will influence the hidden state  $h_t$ ; its formulation is given by:

$$o_t = \sigma(W_o \cdot x_t + U_o \cdot h_{t-1} + O_o \cdot m_{ij} + b_o) \quad (11)$$

The other components of the LSTM architecture remain unchanged. The candidate memory cell, which proposes new values that can be used to update the cell state  $c_t$ , is computed as:

$$\tilde{c}_t = \tanh(W_c \cdot x_t + U_c \cdot h_{t-1} + b_c) \quad (12)$$

The memory cell itself, which is the long-term memory of the network, is updated by:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (13)$$

The hidden state, representing the short-term memory of the network, is then obtained as:

$$h_t = o_t \odot c_t \quad (14)$$

Here,  $x_t$  is the input vector,  $W$  and  $U$  are the weight matrices,  $b$  is the bias vector and  $\sigma$  and  $\tanh$  are the sigmoid and the hyperbolic tangent activation functions.

This model returns three outputs like the classic LSTM: output, hidden\_state and cell\_state. The hidden state ( $\mathbf{S\_T}$ ) was chosen as the output of the module and therefore is one of the inputs of the next module.

#### 4.4. Candidate POI Matching

This is the final module of the model; given the outputs of the local pattern, general pattern and spatio-temporal LSTM modules, the categories of the next POIs are determined. To do this, the module combines its three inputs and extracts the features by using a MLP like:

$$features = \text{concat}([L, G, S\_T]) \cdot W_{features} + b_{features} \quad (15)$$

$$\mathbf{F} = \begin{cases} features & \text{if } features \geq 0 \\ \alpha \cdot features & \text{if } features < 0 \end{cases} \quad (16)$$

Then, the extracted features ( $F$ ) are given as input together with the embedding of the candidate POIs ( $D$ ) to a cross attention module:

$$Q = D \cdot W_Q \quad K = F \cdot W_K \quad V = F \cdot W_V \quad (17)$$

$$output = \text{softmax} \left( \frac{Q \cdot K^T}{\sqrt{d}} \right) \cdot V \quad (18)$$

where  $W$  is the weight matrix and  $d$  is the last dimension of  $Q$ .

Last, through the use of a second MLP, the probabilities of each category for the next POI are determined for each user:

$$\text{next\_POI\_probs} = \sigma(output \cdot W_{final} + b_{final}) \quad (19)$$

where  $\sigma$  is a softmax activation function.

## 5. Experiment

A description of the variables that will be mentioned in this section is given in Table 2.

| Variable       | Description                                                                                              |
|----------------|----------------------------------------------------------------------------------------------------------|
| $lat_C, lon_C$ | The average latitude/average longitude of all POIs in the dataset (the centroid's latitude/longitude).   |
| $t_{Min}$      | The earliest check-in time recorded in the dataset.                                                      |
| $t_{Max}$      | The latest check-in time recorded in the dataset.                                                        |
| $distance$     | The squared Euclidean distance between a POI's location and the centroid.                                |
| $time$         | A normalized time value representing the relative check-in time.                                         |
| $K$            | The optimal number of clusters determined by applying K-Means (e.g., $K = 6$ based on the elbow method). |

**Table 2**  
Description of variables used in the experiment section

#### 5.1. Settings

This subsection shows the hyperparameters used for our experiment. We considered batches of 10 elements, each with sequences of 100 check-ins. For training the model, we used the Adam optimizer, with learning rate and weight decay equal to  $3 \times 10^{-4}$  and  $1 \times 10^{-5}$  respectively; before training, an initialization of the model was performed using the uniform Xavier distribution.

We set the embedding dimensions of user IDs to 10, POI IDs to 20, POI category ID, POI position and cluster to 60, check-in time to 10, candidate POIs to 30 and space-time difference to 60. The hidden dimension of the Local Pattern, General Pattern and Spatio-Temporal LSTM modules was set to 64.

The model was trained for a total of 100 epochs. The dataset used was divided into three parts, forming train set (60%), validation set (20%) and test set (20%): the first two were used in the training phase, while the third one was used for the evaluation of the model.

#### 5.2. Dataset

The Dataset that we used is called *Foursquare Dataset*. It is composed of the user's check-in in **Tokyo** (TKY) and **New York** (NYC) and one sample contains the information about the user  $u$  who visited a POI  $v$ , located in  $p$  at time  $t$ . An example of a sample is shown in table 3.

| Feature         | Value                          |
|-----------------|--------------------------------|
| User Id         | 868                            |
| Venue Id        | 4b7b884ff964a5207d662fe3       |
| Category Id     | 4bf58dd8d48988d1d1941735       |
| Category Name   | Ramen / Noodle House           |
| Latitude        | 35.71558112039315              |
| Longitude       | 139.80031728744507             |
| Timezone Offset | 540                            |
| UTC Time        | Tue Apr 03 18:22:04 +0000 2012 |

**Table 3**  
Example of the features of a sample extracted from the TKY dataset before processing

Before feeding the data into the model, a preprocessing step was performed to ensure consistency and proper formatting. After preprocessing, each sample contains the features shown in table 4, where it is also possible to see an example of the possible processed values.

All IDs are unique identifiers starting from 1. The *Distance* feature represents the Euclidean distance from the geographical centroid (i.e., the mean position of all POIs), calculated as:

$$\text{distance} = \sqrt{(\text{lat} - \text{lat}_C)^2 + (\text{lon} - \text{lon}_C)^2} \quad (20)$$

| Feature     | Value   |
|-------------|---------|
| User ID     | 1920.0  |
| Venue ID    | 51509.0 |
| Category ID | 506.0   |
| Distance    | 0.1431  |
| Time        | 0.8852  |

**Table 4**  
Example of processed sample; the features and the corresponding values are different with respect to the ones of table 3

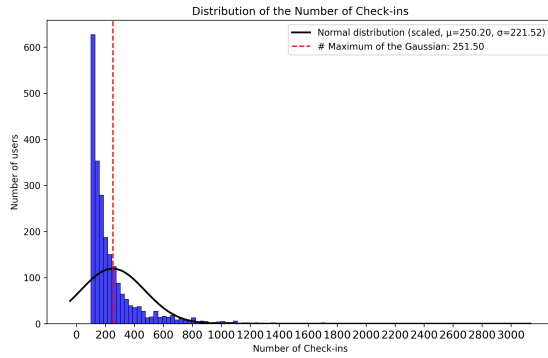
where  $lat_C$  and  $lon_C$  denote the latitude and longitude of the centroid, respectively. The *Time* feature is a normalized value that indicates the relative timestamp of the check-in, computed as:

$$time = \frac{t - t_{Min}}{t_{Max} - t_{Min}} \quad (21)$$

where  $t$  is the timestamp of the current check-in, and  $t_{Min}$  and  $t_{Max}$  are the minimum and maximum timestamps in the dataset, respectively. For our experiment, we chose to use only the Tokyo dataset (TKY); its general statistics are summarized as follows:

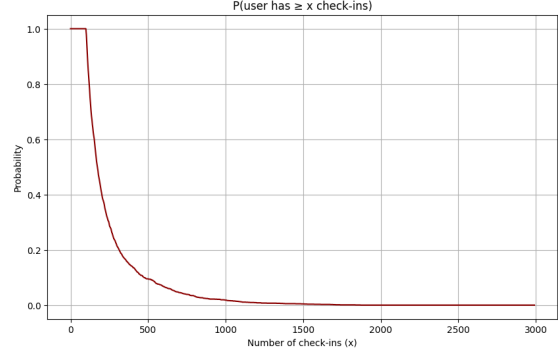
- Number of unique users: 2,293
- Number of unique POIs: 61,858
- Number of unique POI categories: 385
- Maximum number of check-ins by a user: 2,991
- Minimum number of check-ins by a user: 100

The final preprocessing step involves setting a threshold for the maximum number of check-ins allowed per user. To determine this threshold, we analyze the distribution of check-ins across users (fig. 3).



**Figure 3:** Distribution of check-ins per user. Each bin spans approximately 30 check-ins, and most users fall within the range  $100 \leq x \leq 130$ , indicating that the majority have performed that many check-ins.

We then use the percentile-based probability distribution to estimate the likelihood that a given user  $u$  has performed at least  $x$  check-ins (fig. 4).



**Figure 4:** Likelihood that a given User  $u$  has performed at least  $x$  check-ins. For example the probability that a user has performed at least  $\approx 200$  check-in is equal to 0.8

From this analysis, we can infer that, in order to discard only the bottom 10% of users, the threshold should be set to 109. In other words, 90% of the users have made at least 109 check-ins. However, the difference between 109 and 100 is not significant in this context, so the threshold has been set to 100 for simplicity (no user was discarded).

### 5.3. Clusters

During the preprocessing phase of the used dataset, we determined the clusters to which the POIs belong using their position (*latitude, longitude*). To perform this task, we used the K-Means algorithm; however, we had to determine the ideal value of  $K$  first.

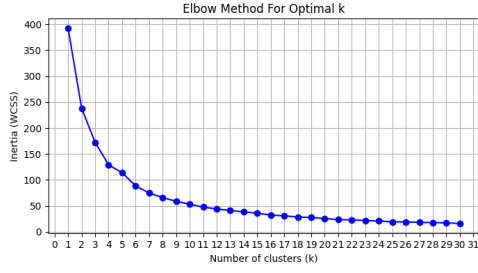
To select  $K$ , we used the elbow method, which evaluates the within-cluster sum of squares (*WCSS*) when  $K$  is changed (in our case in a range between 1 and 30). By visual inspection of the graph in figure 5, we chose the value of  $K$  corresponding to the elbow point, i.e. the point where the best compromise between the number of clusters and the value of the *WCSS* is found.

After determining that the ideal value was  $K = 6$ , K-Means was applied to the set of all POIs; the determined clusters were then assigned to all elements of the sequences in the dataset.

### 5.4. Metrics

To evaluate the performance of the model, we used *recall@K* metric, which is a common choice for the evaluation of recommendation systems. This metric measures the ability of the recommendation system to identify the next correct class among the most likely  $K$ . Its





**Figure 5:** Elbow method plot which contain the within-cluster sum of squares (WCSS) for different numbers of clusters ( $K$ ); it is used to identify the optimal number of clusters

value is between 0 and 1, where the higher the value, the better the model’s performance; this value is calculated using the formula:

$$Recall@K = \frac{\text{Number of Relevant Items in Top } K \text{ Recommendations}}{\text{Total Number of Relevant Items}} \quad (22)$$

### 5.5. Baselines

As a reference for the performance of our trained model, we used seven baselines. CyGNet [12] is a representation learning model for temporal knowledge graphs, which is based on a time-aware copy-generation mechanism. DeepMove [8] is a historical attention recurrent model used for mobility prediction, capable of capturing both short- and long-term mobility patterns from past trajectories. GeoSAN [13] uses a self-attention mechanism and is used for location recommendation tasks. GESTIAN [9] is a graph-based general pattern learning model used to learn common behaviour patterns; it is based on a global trajectory flow graph, which manages the problems caused by the cold start. STAN [14] is a spatio-temporal attention network used for location recommendation; to do it, it uses the relative spatio-temporal information of all the check-ins. STGCAN [15] captures the personalized preferences of the users by modelling the spatio-temporal influence of neighbouring check-ins to recommend the most relevant next point-of-interest. Finally, TiSASRec [16] is based on the self-attention mechanism, to which is added information about the time intervals between user-object interactions; in this way, the temporal dynamics in user behaviour are better captured in order to obtain more accurate recommendations.

## 6. Results

Table 5 shows the experiment results of the proposed model and several state of the art models, all trained and evaluated using the TKY dataset. The  $Recall@K$  metric was used for the evaluation, with  $K = 1, 5, 10$ .

|                  | Recall@1     | Recall@5     | Recall@10    |
|------------------|--------------|--------------|--------------|
| TiSASRec         | 0.098        | 0.227        | 0.369        |
| CyGNet           | 0.109        | 0.294        | 0.406        |
| DeepMove         | 0.124        | 0.268        | 0.351        |
| GeoSAN           | 0.125        | 0.296        | 0.374        |
| STAN             | 0.133        | 0.346        | 0.426        |
| GESTIAN          | 0.148        | <b>0.392</b> | <b>0.507</b> |
| STGCAN           | 0.171        | 0.357        | 0.457        |
| <b>Our Model</b> | <b>0.217</b> | <b>0.315</b> | <b>0.439</b> |

**Table 5**

Recommendation performance comparison with baselines

We can see how the proposed model outperformed all considered baselines in terms of  $Recall@1$  (21.7%). This highlights the effectiveness of the model in identifying the specific class of the next POI.

In terms of  $Recall@5$ , our model performed very well (31.5%), being surpassed only by the STAN (34.6%), STGCAN (35.7%) and GESTIAN (39.2%) models.

Lastly, in the case of  $Recall@10$ , only the STGCAN (45.7%) and GESTIAN (50.7%) models outperformed the proposed model (43.9%).

## 7. Conclusion

In summary, this work presents a new model that improves the prediction of the next POI category by capturing both local and general user movement patterns. Our method uses graph convolution to catch motions within a group and graph attention to track movements between different groups. We also include a spatio-temporal LSTM to follow order and timing of user check-ins. Finally, our candidate POI matching section combines all these features to provide better recommendations.

On the Foursquare Tokyo dataset, our model shows competitive performance, especially for  $Recall@1$ , which outperforms other baselines. One main reason we see such high  $Recall@1$  is our effective use of clustering. By dividing POIs into groups, the model reduces the search area and focuses only on the spots most likely to contain the next POI, which raises the chance of selecting the right one.

To sum up, our new model shows that by combining information about local and general movement patterns, we can better predict the next POI category. By clustering

POIs into groups and using graph convolution, graph attention, and a spatio-temporal LSTM, the model focuses on the most promising areas, which is one key reason for its strong performance, especially in *Recall@1*. For future work, we plan to include more types of information, like user profiles and social data, to further enhance recommendation accuracy. We also aim to test our model on other datasets and in different city environments to see how well it works across various real world scenarios.

## References

- [1] L. M, B. G, L. B, P. L, A survey on deep learning for human mobility (2023). doi:10.1145/3485125and10.48550/arxiv.2012.02825.
- [2] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Computation* 9 (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.
- [3] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *International Conference on Learning Representations (ICLR)* (2017). URL: <https://arxiv.org/abs/1609.02907>.
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, 2018. URL: <https://arxiv.org/abs/1710.10903>. arXiv:1710.10903.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, 2023. URL: <https://arxiv.org/abs/1706.03762>. arXiv:1706.03762.
- [6] B. Chang, Y. Park, D. Park, S. Kim, J. Kang, Content-aware hierarchical point-of-interest embedding model for successive poi recommendation, in: *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18, International Joint Conferences on Artificial Intelligence Organization*, 2018, pp. 3301–3307. URL: <https://doi.org/10.24963/ijcai.2018/458>. doi:10.24963/ijcai.2018/458.
- [7] S. Zhang, H. Tong, J. Xu, R. Maciejewski, Graph convolutional networks: a comprehensive review, *Computational Social Networks* 6 (2019) 11. URL: <https://doi.org/10.1186/s40649-019-0069-y>. doi:10.1186/s40649-019-0069-y.
- [8] J. Feng, Y. Li, C. Zhang, F. Sun, F. Meng, A. Guo, D. Jin, Deepmove: Predicting human mobility with attentional recurrent networks, in: *Proceedings of the 2018 World Wide Web Conference, WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE*, 2018, p. 1459–1468. URL: <https://doi.org/10.1145/3178876.3186058>. doi:10.1145/3178876.3186058.
- [9] X. Zhang, D. Liu, L. Yan, Z. Zhang, Y. Li, Graph-enhanced spatio-temporal interval aware network for next poi recommendation in mobile environment, *Journal of Internet Technology* 25 (2024) 619–628. URL: <https://jit.ndhu.edu.tw/article/view/3079>.
- [10] L. Huang, Y. Ma, S. Wang, Y. Liu, An attention-based spatiotemporal lstm network for next poi recommendation, *IEEE Transactions on Services Computing* 14 (2021) 1585–1597. doi:10.1109/TSC.2019.2918310.
- [11] P. Zhao, H. Zhu, Y. Liu, Z. Li, J. Xu, V. S. Sheng, Where to Go Next: A Spatio-temporal LSTM model for Next POI Recommendation, *arXiv e-prints* (2018) arXiv:1806.06671. doi:10.48550/arXiv.1806.06671. arXiv:1806.06671.
- [12] C. Zhu, M. Chen, C. Fan, G. Cheng, Y. Zhang, Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks, *Proceedings of the AAAI Conference on Artificial Intelligence* 35 (2021) 4732–4740. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/16604>. doi:10.1609/aaai.v35i5.16604.
- [13] D. Lian, Y. Wu, Y. Ge, X. Xie, E. Chen, Geography-aware sequential location recommendation, in: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, Association for Computing Machinery, New York, NY, USA*, 2020, p. 2009–2019. URL: <https://doi.org/10.1145/3394486.3403252>. doi:10.1145/3394486.3403252.
- [14] Y. Luo, Q. Liu, Z. Liu, Stan: Spatio-temporal attention network for next location recommendation, in: *Proceedings of the Web Conference 2021, WWW '21, Association for Computing Machinery, New York, NY, USA*, 2021, p. 2177–2185. URL: <https://doi.org/10.1145/3442381.3449998>. doi:10.1145/3442381.3449998.
- [15] X. Wang, G. Sun, X. Fang, J. Yang, S. Wang, Modeling spatio-temporal neighbourhood for personalized point-of-interest recommendation, in: L. D. Raedt (Ed.), *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22, International Joint Conferences on Artificial Intelligence Organization*, 2022, pp. 3530–3536. URL: <https://doi.org/10.24963/ijcai.2022/490>. doi:10.24963/ijcai.2022/490, main Track.
- [16] J. Li, Y. Wang, J. McAuley, Time interval aware self-attention for sequential recommendation, in: *Proceedings of the 13th International Conference on Web Search and Data Mining, WSDM '20, Association for Computing Machinery, New York, NY, USA*, 2020, p. 322–330. URL: <https://doi.org/10.1145/3336191.3371786>. doi:10.1145/3336191.3371786.